

Sky Analytics Engine

Analisi batch e Metriche del Traffico Aereo

Flavio Simonelli Francesco Masci

Corso Architettura e Sistemi per Big Data
Corso di Laurea Magistrale in Ingegneria Informatica
Università di Roma "Tor Vergata"

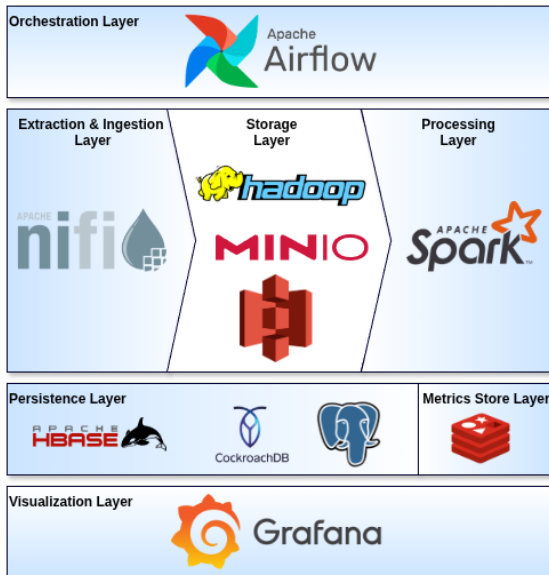
12 Giugno 2026



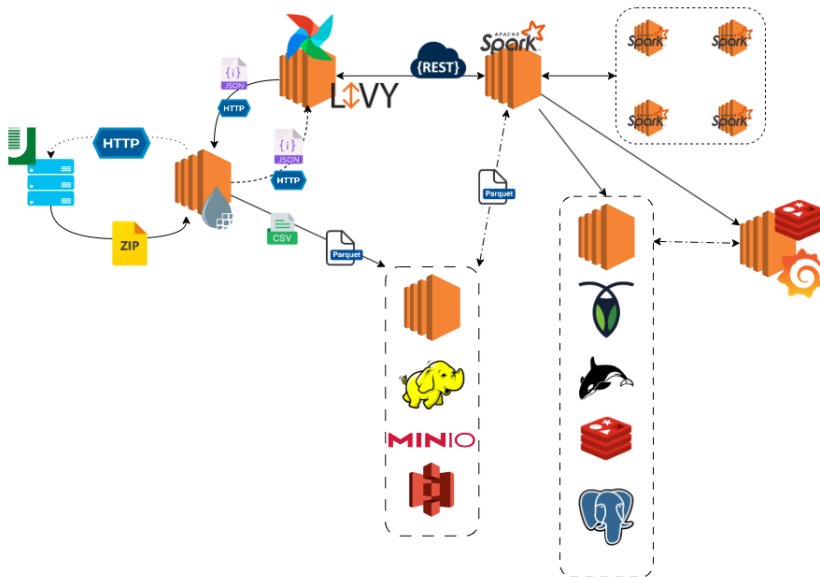
TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

- 1 Big Data Stack
- 2 Flow Diagram
- 3 Sequence Diagram
- 4 Extraction & Ingestion Layer
- 5 Deployment
- 6 Computation Layer
- 7 Airflow Orchestration
- 8 Valutazione Sperimentale: Metriche

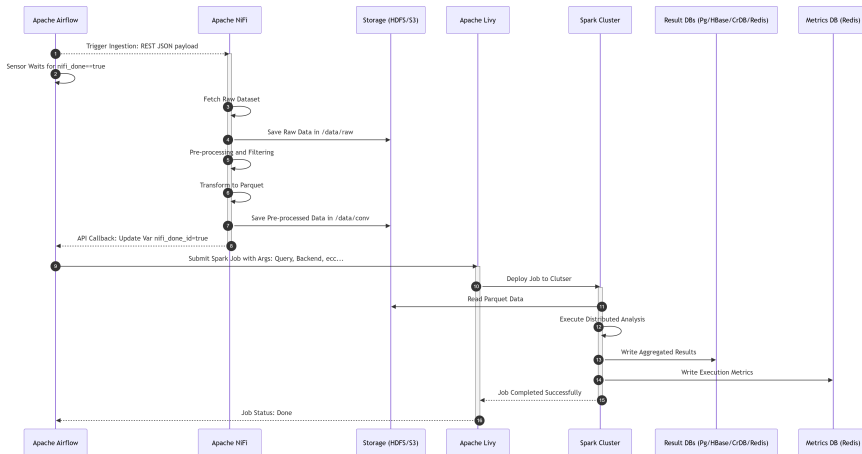
Big Data Stack



Flow Diagram



Sequence Diagram

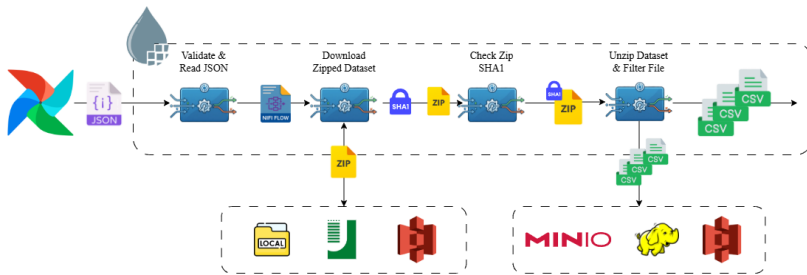


NiFi Ingestion Pipeline: Approccio Data-Driven

- **Design Data-Driven:** Pipeline stateless configurata dinamicamente tramite payload JSON iniettato all'avvio (EvaluateJsonPath).
- **Parametri chiave del JSON:**
 - `experiment.id`: Identificativo univoco del job.
 - `ingestion`: Parametri sorgente (URL HTTP, SHA-1 atteso).
 - `storage`: Percorsi dinamici per dati Raw e Preprocessed (HDFS/S3).
 - `processing.strategy`: Scelta della strategia di ottimizzazione (PARTITION PRUNING o PREDICATE PUSHDOWN).
 - `callback`: Configurazione per la notifica finale ad Airflow.

Ingestion, Sicurezza e Gateway

- **Estrazione:** Ricezione e spaccettamento degli archivi (UnpackContent).
- **Validazione SHA-1:**
 - Controllo di integrità crittografica sui file estratti.
 - Confronto hash atteso vs calcolato (CryptographicHashContent).
 - Routing degli errori per isolare dati corrotti.
- **Gateway & Biforcazione:**
 - **Ramo Raw:** Salvataggio as-is per garanzia del dato originale.
 - **Ramo Preprocessing:** Inoltro dei record validati alla trasformazione.



Preprocessing e Ottimizzazione in Parquet

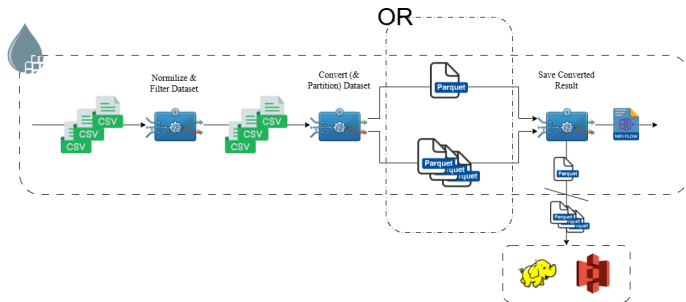
- **Operatori NiFi per la Trasformazione:**

- QueryRecord/UpdateRecord: Filtraggio, casting e normalizzazione.
- LookupRecord: Arricchimento dei dati via file dizionario.
- MergeRecord: Prevenzione dello "Small Files Problem".

- **Target Parquet:** Conversione con schema enforcement (Avro).

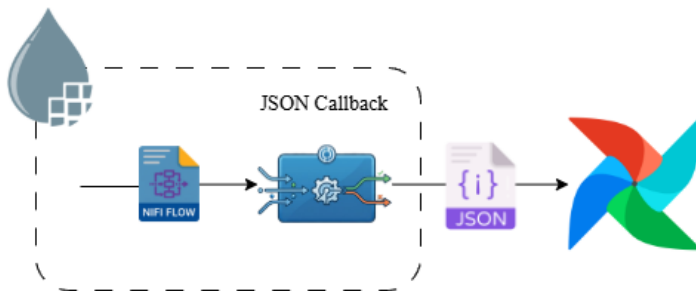
- **Strategie di Ottimizzazione:**

- **Partition Pruning:** Generazione dinamica dei path (UpdateAttribute).
- **Predicate Pushdown:** Ottimizzazione dei metadati per Spark.

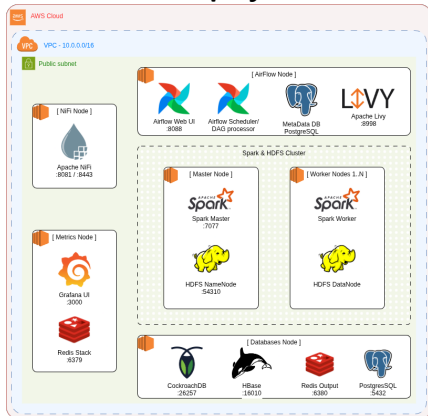


Storage Finale e Handover ad Airflow

- **Storage Distribuito:** Scrittura dei file Parquet ottimizzati in HDFS o S3.
- **Notifica di Completamento:**
 - Chiamata REST via InvokeHTTP (PATCH/POST) alle API di Airflow.
 - Handover del payload aggiornato con lo stato dell'elaborazione.
- **Orchestrazione Analitica:** Sblocco automatico dei successivi job Spark in base alla strategia scelta.



EC2 Deployment



EMR Deployment

[Immagine in arrivo]

- **Ecosistema Spark:** Implementazione in Java utilizzando Spark Core (RDD), Spark SQL e MLlib.
- **Design Pattern Template Method:**
 - Astrazione tramite la classe `BaseQuery.java`.
 - Garanzia di coerenza tra i diversi paradigmi di esecuzione.
- **Benchmarking Comparativo:** Ogni analisi implementa obbligatoriamente tre motori:
 - **RDD API:** Controllo granulare su partizioni e serializzazione.
 - **DataFrame API:** Astrazione strutturata (DSL) e ottimizzazione *Catalyst*.
 - **Spark SQL:** Interfaccia dichiarativa per logiche di business complesse.

Query 1: Analisi Mensile delle Performance

- **Obiettivo:** Ritardi medi, estremi e tasso di cancellazione mensile per vettore.
- **RDD (aggregateByKey):**
 - `pairs.aggregateByKey(zeroValue, seqOp, combOp)`
 - Aggregazione locale su partizione per minimizzare il traffico di rete.
- **DataFrame (DSL):**
 - `flights.groupBy("MONTH", "CARRIER").agg(avg("DELAY"), ...)`
- **Spark SQL:**
 - `SELECT MONTH, AVG(DELAY) FROM flights GROUP BY MONTH, ...`

Query 2: Ranking dei Ritardi in Arrivo

- **Obiettivo:** Identificazione delle Top-K tratte più critiche.
- **RDD (Manuale):**
 - `mapToPair().reduceByKey().takeOrdered(10, comparator)`
- **DataFrame (Window Functions):**
 - `agg(avg("ARR_DELAY")).orderBy(col("mean").desc()).limit(10)`
- **Spark SQL:**
 - `SELECT carrier, AVG(ARR_DELAY) as mean FROM flights ...
ORDER BY mean DESC LIMIT 10`

Query 3: Distribuzione e Percentili

- **Obiettivo:** Statistiche orarie (Q1, Mediana, Q3).
- **RDD (La sfida):** `groupByKey()` su milioni di record causerebbe OOM.
- **DataFrame / SQL (Approssimazione):**
 - `agg(expr("percentile_approx(DEP_DELAY, 0.5)))`
 - Uso dell'algoritmo nativo Greenwald-Khanna per stime efficienti.
- **Soluzione RDD Custom:**
 - `PercentileSketch sketch = PercentileSketch.from(config)`
 - Integrazione di T-Digest e KLL per calcolo distribuito e mergeable.

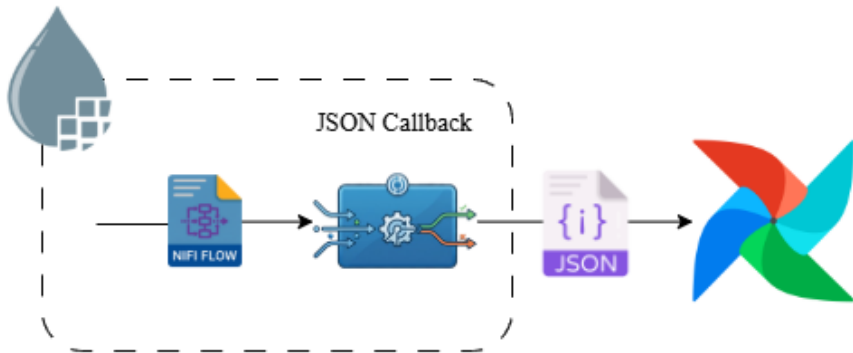
- **Design Pattern Strategy:** Implementazione di algoritmi probabilistici in RDD.
- **T-Digest Strategy:**
 - Struttura dati basata su centroidi pesati.
 - Alta precisione nelle "code" della distribuzione (ritardi estremi).
- **KLL Sketch Strategy:**
 - Algoritmo stream-based con footprint di memoria fisso e predicibile.
 - Merge delle strutture dati tra worker senza trasferimento di interi dataset.

- **Obiettivo:** Raggruppamento di vettori aerei con profili operativi simili.
- **Modello K-Means** (*Spark MLlib*):
 - Inizializzazione tramite `k-means||` (variante parallela di `k-means++`).
 - Convergenza distribuita sui centroidi dei cluster.
- **Data Pipeline:**
 - **VectorAssembler:** Consolidamento delle features analitiche.
 - **StandardScaler:** Normalizzazione fondamentale per la distanza Euclidea.

- **Obiettivo:** Misurazione rigorosa dei tempi di esecuzione per il benchmarking.
- **JobTimerListener:**
 - Estensione di `SparkListener` per intercettare gli eventi del DAG Scheduler.
 - Monitoraggio di `onJobStart` e `onJobEnd`.
- **Precisione:** Eliminazione degli overhead di sistema, I/O Driver e task "lazy", garantendo metriche focalizzate puramente sul tempo di calcolo distribuito.

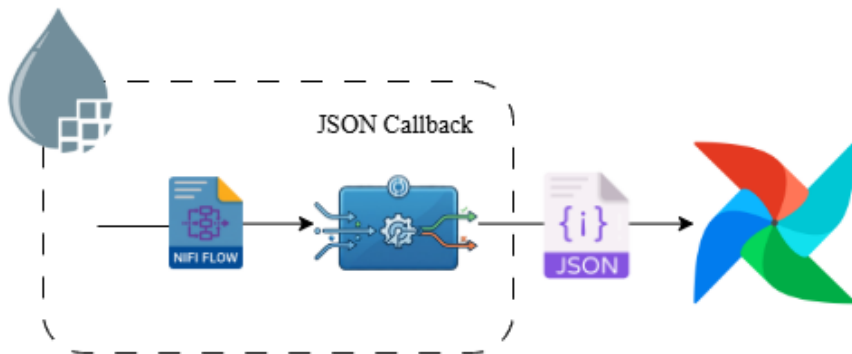
Orchestrazione End-to-End: flight_analysis

- **Obiettivo:** Orchestrare l'intera pipeline, dall'ingestion in NiFi all'elaborazione Spark.
- **Funzionalità:** Utilizzo della **TaskFlow API** (Airflow 3.0) per gestire il branching condizionale (*Ingest Only, Execution Only, Both*).
- **Sincronizzazione:** Uso di `Task.Sensor` per attendere il completamento asincrono di NiFi.



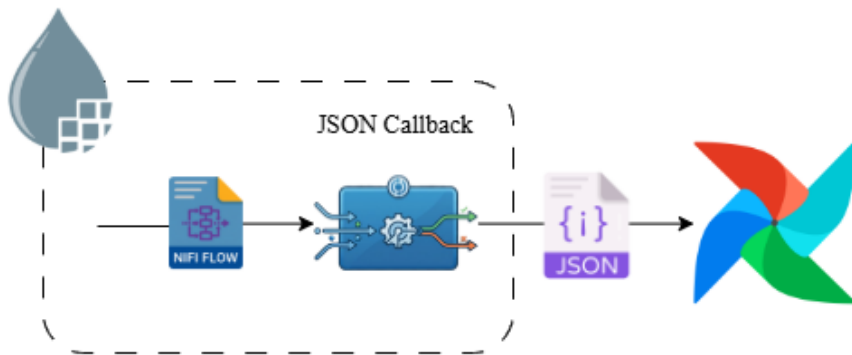
Benchmarking su Cluster Self-Hosted: ec2_benchmark

- **Obiettivo:** Raccolta automatizzata delle metriche prestazionali in ambiente EC2.
- **Logica:** Loop sequenziale per eseguire tutte le 12 combinazioni (4 Query $\times$ 3 Backend).
- **Integrazione:** Sottomissione dei job tramite LivyOperator verso il Master Node.



Benchmarking su AWS Managed: `emr_benchmark`

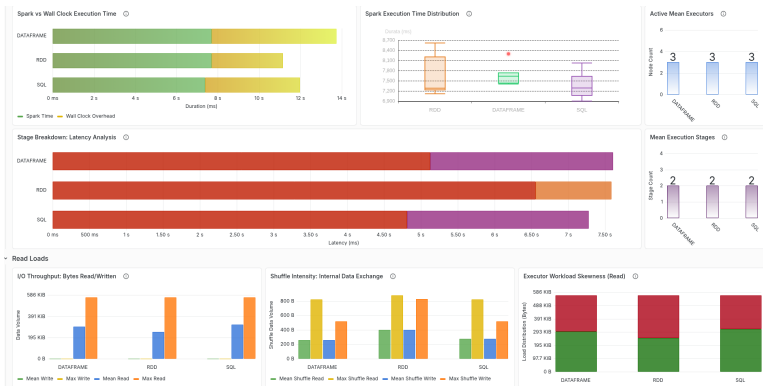
- **Obiettivo:** Benchmarking su cluster **AWS EMR** per testare la scalabilità managed.
- **Logica:** Esecuzione delle 12 combinazioni analitiche in modalità sequenziale.
- **Integrazione Cloud:** Uso di `EmrAddStepsOperator` per l'injection dei job e `EmrStepSensor` per il monitoraggio dello stato degli step.



- **Rigore Statistico:** Ogni combinazione (Query $\times$ Backend) è stata eseguita **5 volte** per garantire la stabilità delle misurazioni.
- **Telemetria:** Utilizzo del JobTimerListener (SparkListener) per isolare il *tempo netto di calcolo* dall'overhead di sistema.
- **Hardware e Infrastruttura:**
 - **EC2 (Self-Hosted):** Master + Worker su istanze t3.small. Storage basato su HDFS.
 - **EMR (Cloud-Native):** 1 Master + 3 Core su istanze m6g.xlarge (ARM64). Architettura *Shared-Nothing* su Amazon S3 via s3a://.

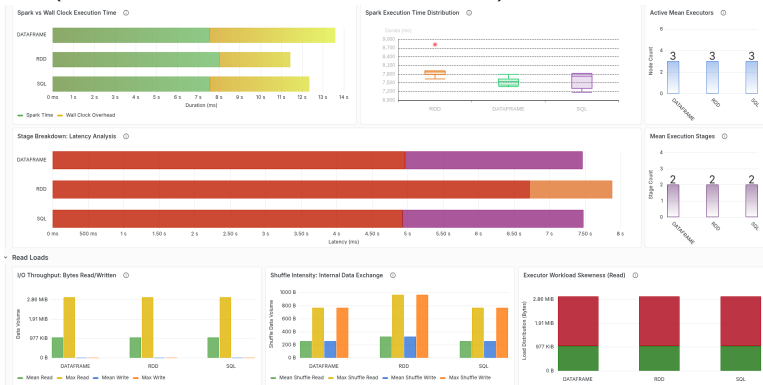
Confronto API: RDD vs DataFrame vs SQL

- **DataFrame/SQL**: Massima stabilità e minori tempi di esecuzione nelle query di aggregazione (es. Q1) grazie all'ottimizzatore *Catalyst* e alla gestione binaria off-heap di *Tungsten*.
- **RDD**: Più performante in task atomici e logiche distribuite custom (Q2 e Q3), nonostante il maggior overhead di serializzazione JVM.



Ottimizzazione I/O: Pruning vs Pushdown

- **Partition Pruning:** Strategia ottimale. Esclude i dati a livello di file system, garantendo un I/O bilanciato e l'assenza di *Workload Skew* sui nodi.
- **Predicate Pushdown:** Richiede la scansione dei metadati Parquet. Genera un volume di lettura superiore e tende a sovraccaricare singoli nodi (colli di bottiglia in fase di scansione).



Overhead Infrastrutturale: EC2 vs EMR

- **Standalone (EC2):** Overhead di *Wall Clock Time* significativo dovuto ai tempi di negoziazione delle risorse, bootstrap del cluster e latenze di rete HDFS.
- **Managed (EMR):** L'integrazione nativa con S3 e la gestione elastica delle istanze contraggono sensibilmente i "tempi morti", massimizzando l'efficienza temporale del job.

